Now we can combine comparison-based predicates with these operators to form more complex expressions such as

```
$a = 1;
$b = 2;
if ($a < $b && $b === 2) {
    print 'A < B AND B = 2' . PHP_EOL;
}
if ($a > $b || $b === 2) {
    print 'A > B OR B = 2' . PHP_EOL;
}
```

```
Output:
A < B AND B = 2
A > B OR B = 2
```

Hopefully you get the idea: the first expression has both predicates evaluate to `true`, and we've used logical && (AND); the second expression has only the right-hand side, which is `true`, but this time, we've used logical || (OR), which only requires one side to be true. It is perfectly fine to chain together many predicates, and parentheses can be included to improve readability:

```
if (
    (someFunction() === 'foo' && someOtherFunction() === 'bar')
    || fallbackFunction() === 'qux'
) {
    // whatever
}
```

Adding parentheses to a logical expression also impacts the precedence of operations – just like in mathematics – which we'll cover shortly.

Short-Circuiting

PHP will exit logical expressions early if the results are conclusive before the entire expression has been evaluated, as the following demonstrates: